

基于 MAUI 的视频目标识别系统设计与实现

摘要：随着深度学习技术的快速发展，目标检测技术在视频监控、自动驾驶、智能交通等领域得到了广泛应用。传统的目标检测系统往往依赖于特定的硬件平台或专用的移动应用，难以实现跨平台部署和便捷的用户体验。本文设计并实现了一个基于.NET MAUI框架的视频目标识别系统。它采用前后端分离的分布式架构。前端使用MAUI技术实现跨平台应用，后端采用Flask框架提供RESTful API服务，核心检测算法采用YOLOv11深度学习模型。系统主要功能包括视频文件上传、实时目标检测、视频流推送、检测结果存储与展示等。前端MAUI应用实现了视频列表管理、视频上传、实时处理状态和视频播放功能；后端Flask服务实现了视频上传接口、处理状态查询、MJPEG视频流推送等API。系统采用MySQL数据库存储视频信息和检测结果，使用OpenCV进行视频帧处理。针对CPU推理环境下的性能优化需求，本文采用了跳帧处理策略、检测结果缓存、置信度阈值过滤等优化措施，有效提升了系统的处理速度。测试结果表明，在720P视频输入条件下，系统CPU推理速度可达15帧/秒以上，满足实时处理的需求。

关键词：MAUI, 目标检测, YOLOv11, Flask, 实时处理, MJPEG视频流

中图分类号：TP311.52

Design and Implementation of Video Target Recognition System

Based on MAUI

Abstract:With the rapid development of deep learning technology, object detection technology has been widely used in video surveillance, automatic driving, intelligent transportation and other fields. Traditional target detection systems often rely on specific hardware platforms or dedicated mobile applications, making it difficult to achieve cross-platform deployment and convenient user experience. This paper designs and implements a video target recognition system based on. NET MAUI framework. It adopts a distributed architecture with front-end and back-end separation. The front-end uses MAUI technology to implement cross-platform applications, the back-end uses Flask framework to provide RESTful API services, and the core detection algorithm uses YOLOv11 deep learning model. The main functions of the system include video file upload, real-time target detection, video stream push, detection result storage and display. The front-end MAUI application implements video list management, video upload, real-time processing status and video playback functions ; the back-end Flask service implements APIs such as video upload interface, processing status query, MJPEG video stream push, etc. The system uses MySQL database to store video information and detection results, and uses OpenCV for video frame processing. Aiming at the performance optimization requirements in the CPU inference environment, this paper adopts optimization measures such as frame skipping processing strategy, detection result caching, and confidence threshold filtering, which effectively improves the processing speed of the system. The test results show that the CPU inference speed of the system can reach more than 15 frames per second under the condition of 720P video input, which meets the needs of real-time processing.

Keywords: MAUI , target detection , YOLOv11 , Flask , real-time processing , MJPEG video stream

Classification: TP311.52

目次

摘要	I
目次	III
1. 引言	1
1.1. 研究背景与意义	1
1.2. 国内外研究现状	1
1.3. 研究内容与目标	2
2. 相关技术介绍	3
2.1. .NET MAUI 框架	3
2.2. Flask Web 框架	3
2.3. YOLOv11 目标检测算法	4
2.4. MySQL 数据库	5
2.5. OpenCV 视觉库	5
2.6. MJPEG 视频流技术	5
3. 需求分析与系统设计	7
3.1. 系统需求分析	7
3.1.1. MJPEG 视频流技术	7
3.1.2. 功能需求分析	7
3.1.3. 性能需求分析	8
3.2. 系统架构设计	8
3.3. 数据库设计	9
3.3.1. 数据库概念设计	9
3.3.2. 数据库表结构设计	10
3.3.3. 接口设计	11
4. 系统实现	12
4.1. 前端 MAUI 应用实现	12
4.2. 后端 Flask 服务实现	16

4.3. 性能优化实现	18
5. 系统测试	20
5.1. 测试环境	20
5.2. 功能测试	20
5.2.1. 视频上传功能测试	20
5.2.2. 实时目标检测功能测试	20
5.2.3. 结果展示功能测试	21
5.3. 测试总结	21
6. 总结与展望	22
6.1. 工作总结	22
6.2. 工作创新点	22
6.3. 不足与展望	23
参考文献	24
学位论文数据集	25

1. 引言

1.1. 研究背景与意义

这些年人工智能技术进步得很快，计算机视觉领域也跟着有了不少突破。其中，目标检测技术作为计算机视觉的核心任务之一，在视频监控^[1]、自动驾驶^[2]、智能交通^[3]、人脸识别^[4]这些场景里，正发挥着越来越重要的作用。

最近几年，深度学习在目标检测方面也取得了很不错的成果。从早期的R-CNN^[5]、Fast R-CNN^[6]，到后来的YOLO系列算法^[7-8]，检测的精度和速度都在不断提高。特别是YOLO（You Only Look Once）系列算法，因为检测速度快、精度也不错，已经成为实时目标检测领域的主流选择。YOLOv11作为YOLO系列的最新版本，在模型结构和训练策略上进行了多项优化，其检测性能又上了一个台阶^[9]。

再看应用层面。传统的目标检测系统，要么得部署在特定的硬件平台上，要么就得用原生的移动开发技术（比如iOS上的Swift或者Android上的Kotlin）分别单独开发应用。这不仅增加了开发成本，而且跨平台部署也很麻烦。.NET MAUI（Multi-platform App UI）作为一种比较新的跨平台开发框架，允许开发者使用单一的C#代码库，就能构建出原生的Android、iOS、Windows、macOS应用，大大简化了跨平台应用的开发和维护工作^[10]。

基于这样的背景，本文设计并实现了一个基于MAUI的Windows平台视频目标识别系统。这个系统把MAUI框架和深度学习技术结合在一起，实现了跨平台的目标检测应用，既有研究意义，也有实际应用价值。它给移动端的目标检测应用提供了一种新的开发思路；同时，系统的设计与实现也能为相关领域的研究提供一些参考；其中采用的性能优化策略，对类似系统的开发也有一定的借鉴作用。

1.2. 国内外研究现状

目标检测技术发展了很多年，进步确实不小。从算法上看，卷积神经网络（CNN）的引入大大提高了检测的精度。早在2014年，Girshick等人就提出了R-CNN算法，这是第一次把深度学习用到目标检测上，在Pascal VOC数据集上取得了很明显的性能提升^[5]。随后，Fast R-CNN、Faster R-CNN等改进版本又进一步提升了检测的精度和速度^[6]。

到了2016年，Redmon等人提出了YOLO算法，它的思路与传统不同，把目标检测问题重新做成了一个单一的回归问题，实现了端到端的检测^[7]。大的特点就是速度快，很快就成了实时目标检测领域的热门选择。再往后，YOLOv2、

YOLOv3、YOLOv4、YOLOv5、YOLOv8、YOLOv11等多版本相继发布，在检测精度和速度上不断优化^[8-9]。

从应用层面来看，基于深度学习的目标检测技术已经在不少领域应用成功了。比如在视频监控领域，智能视频分析系统可以自动检测和跟踪感兴趣的目标，监控效率因此提高了不少^[11]；自动驾驶方面，目标检测技术用于识别道路上的车辆、行人、交通标志等，为自动驾驶决策提供依据^[2]；智能交通领域，像车牌识别、违章检测这些技术也得到了广泛应用^[3]。

在移动端目标检测方面，研究人员也在不断探索新的解决方案。传统的移动端目标检测主要依赖于将训练好的模型部署到移动设备上，通过模型压缩、量化等技术降低计算复杂度^[12]。近年来，一些轻量级的目标检测模型如YOLOv5n、YOLOv8n等，专门针对移动端和边缘设备进行了优化，在保证一定精度的前提下大幅降低了计算量和内存占用^[9]。

在跨平台开发框架方面，.NET MAUI作为Xamarin.Forms的进化版本，提供了更现代的架构和更好的开发体验。MAUI采用单项目多平台的方式，允许开发者使用统一的代码库构建面向不同平台的应用^[10]。与Flutter、React Native等其他跨平台框架相比，MAUI与原生平台的关系更加紧密，可以直接调用原生平台的所有功能，具有更好的性能和更强的平台特性支持。

1.3. 研究内容与目标

系统架构设计方面采用前后端分离的分布式架构，设计合理的模块划分和接口规范。前端使用MAUI框架实现跨平台应用，后端使用Flask框架提供RESTful API服务，数据库采用MySQL，实现数据持久化存储。

核心功能要实现视频上传、目标检测、视频流推送、结果存储等核心功能。前端实现视频列表管理、上传界面、实时处理状态监控、视频播放等用户功能；后端实现视频上传接口、处理状态查询、MJPEG视频流推送等API。

性能优化研究方面针对CPU推理环境下的性能需求，研究并实现多种优化策略，包括跳帧处理、检测结果缓存、置信度阈值过滤、输入尺寸优化等，提升系统的处理速度。

本文的研究目标是设计并实现一个基于MAUI的视频目标识别系统，该系统能够实现视频上传、实时目标检测、结果展示等功能，在720P视频输入条件下达到15帧/秒以上的处理速度。

2. 相关技术介绍

2.1. .NET MAUI 框架

.NET MAUI (Multi-platform App UI) 是微软推出的一种跨平台开发框架，用于使用C#和.NET构建原生移动和桌面应用程序。MAUI是Xamarin.Forms的进化版本，于2022年正式发布，相比Xamarin.Forms，MAUI提供了更现代的架构和更好的开发体验^[10]。

MAUI的核心特点是单项目多平台架构。在MAUI中，开发者可以创建一个共享的项目，然后为目标平台（如Android、iOS、Windows、macOS）编译为原生应用。这种架构使得代码复用率达到最大化，同时仍然能够访问每个平台的特定功能和特性。MAUI使用Microsoft的.NET平台作为基础，提供了对平台原生UI的完全访问能力，开发者可以使用C#编写所有平台逻辑，无需编写平台特定代码。

在MAUI中，页面(Page)和布局(Layout)是构建用户界面的基本组件。MAUI提供了丰富的UI控件，如Button、Label、Entry、ListView、CollectionView等，可以满足各种应用场景的需求。MAUI还支持数据绑定(Data Binding)和命令(Command)模式，使得视图与业务逻辑的分离更加便捷^[10]。

MAUI采用了MVVM(Model-View-ViewModel)架构模式来组织代码。Model层负责数据模型和业务逻辑，View层负责用户界面展示，ViewModel层负责连接View和Model，处理用户交互和维护界面状态。CommunityToolkit.Mvvm是微软官方推荐的MVVM辅助库，提供了ObservableProperty、RelayCommand等特性，可以大幅简化MVVM代码的编写^[13]。

在依赖注入方面，MAUI内置了对Microsoft.Extensions.DependencyInjection的支持。通过在MauiProgram.cs中配置依赖注入容器，可以方便地管理应用中各种服务的生命周期，如单例(Singleton)、瞬态(Transient)等。这种设计使得代码更加模块化，测试更加便捷^[10]。

2.2. Flask Web 框架

Flask是一个轻量级的Python Web框架，由Armin Ronacher开发，于2010年首次发布。Flask核心简洁，通过扩展(Extension)的方式提供丰富的功能，是Python生态中最受欢迎的Web框架之一^[14]。

Flask的核心设计理念是“微框架”，它只提供Web应用所需的最基本功能，其他功能如数据库抽象层、表单验证、用户认证等，都通过扩展来实现。这种设计使得Flask非常灵活，开发者可以根据需要选择使用哪些扩展，避免了框架的过度封装。

Flask使用Werkzeug作为WSGI工具库，Jinja2作为模板引擎。Werkzeug提供了请求（Request）和响应（Response）对象、URL路由、调试工具等功能；Jinja2提供了模板继承、宏定义、过滤器等模板功能，使得动态HTML页面的生成变得简单^[14]。

在Flask中，路由（Route）是处理HTTP请求的核心机制。开发者使用@app.route()装饰器来定义URL路径和HTTP方法的映射关系。当客户端请求到达时，Flask会根据URL找到对应的处理函数，并将请求交给该函数处理。处理函数可以访问请求数据、执行业务逻辑、返回响应内容^[14]。

Flask的扩展生态非常丰富。针对不同的需求，有相应的扩展可以使用。对于本系统而言，flask-cors扩展用于处理跨域请求，mysql-connector-python用于连接MySQL数据库，Flask提供了Response对象用于生成流式响应。这些扩展的组合使用，使得Flask能够满足各种Web应用的开发需求。

2.3. YOLOv11 目标检测算法

YOLOv11是YOLO（You Only Look Once）目标检测算法的最新版本，由Ultralytics公司于2024年发布。YOLO系列算法以其高效的检测速度和良好的精度平衡而著称，被广泛应用于实时目标检测场景^[9]。

YOLO算法的核心思想是将目标检测任务重新建模为单一的回归问题。与两阶段检测器（如R-CNN系列）不同，YOLO采用单阶段检测方式，直接在输入图像上预测边界框的位置和类别概率。这种设计使得YOLO的检测速度远超两阶段方法，非常适合实时应用场景。

YOLOv11在之前版本的基础上进行了多项改进。在模型结构方面，YOLOv11采用了更高效的骨干网络（Backbone）和特征金字塔网络（FPN），提升了特征提取和多尺度检测能力。YOLOv11还引入了新的注意力机制，增强了模型对关键特征的感知能力^[9]。

在训练策略方面，YOLOv11采用了数据增强、多尺度训练、标签平滑等技术，提升了模型的泛化能力。YOLOv11还提供了多种预训练模型，包括YOLOv11n（nano）、YOLOv11s（small）、YOLOv11m（medium）、YOLOv11l（large）、YOLOv11x（extra-large）等不同规模，开发者可以根据实际需求选择合适的模型^[9]。

Ultralytics提供了官方的Python库ltralytics，用于加载和运行YOLO模型。该库提供了简洁的API，支持模型推理、模型训练、模型导出等功能。使用model(frame, device='cpu')即可在CPU上执行推理，device='cuda'则使用GPU推理^[15]。

2.4. MySQL 数据库

MySQL是一种开源的关系型数据库管理系统，由瑞典MySQL AB公司开发，目前属于Oracle公司产品。MySQL是Web应用领域最流行的数据库之一，以其高性能、高可靠性、易用性著称^[16]。

MySQL采用客户端-服务器架构。数据库服务器（mysqld）负责管理数据库文件和执行SQL语句，客户端程序负责与服务器通信并发送请求。MySQL支持标准的SQL（Structured Query Language）语句，包括数据定义（DDL）、数据操作（DML）、数据查询（DQL）和数据控制（DCL）四大类语句^[16]。

在数据存储方面，MySQL支持多种存储引擎，如InnoDB、MyISAM、MEMORY等。InnoDB是MySQL 5.5以后的默认存储引擎，支持事务处理、行级锁定、外键约束等高级特性，适合需要高并发和数据一致性要求的应用场景^[16]。

MySQL的连接器（Connector）提供了多种编程语言的数据库访问接口。对于Python应用，可以使用mysql-connector-python或pymysql等库连接MySQL数据库。这些库遵循Python DB-API规范，提供了连接对象（Connection）、游标对象（Cursor）等标准接口，可以方便地执行SQL语句和获取查询结果^[17]。

2.5. OpenCV 视觉库

OpenCV（Open Source Computer Vision Library）是一个开源的计算机视觉和机器学习软件库，由Intel公司于1999年首次发布。OpenCV提供了丰富的图像处理和计算机视觉算法，是计算机视觉领域最流行的开发工具之一^[18]。

OpenCV的核心功能包括图像读取与显示、图像变换、图像滤波、边缘检测、轮廓检测、特征检测与匹配、目标检测等。OpenCV还提供了视频分析功能，包括视频读取与写入、帧处理、运动检测、目标跟踪等^[18]。

在视频处理方面，OpenCV的cv2.VideoCapture类提供了访问摄像头和视频文件的能力，可以逐帧读取视频内容；cv2.VideoWriter类提供了视频写入功能，可以将处理后的帧保存为视频文件。OpenCV支持多种视频编解码器，如MP4V、AVI、XVID等^[18]。

OpenCV的Python接口（cv2模块）提供了与C++接口一致的功能，同时保持了Python的简洁性。在YOLO目标检测应用中，OpenCV用于读取视频帧、将检测结果绘制在图像上、编码输出视频等任务^[18]。

2.6. MJPEG 视频流技术

MJPEG（Motion JPEG）是一种视频压缩格式，将视频的每一帧独立压缩为JPEG图像，然后按顺序播放。MJPEG的优点是实现简单、兼容性好、画质无损

（每帧独立压缩），缺点是压缩比较低，文件体积较大^[19]。

在Web视频流领域，MJPEG通常通过HTTP协议传输，使用multipart/x-mixed-replace MIME类型实现。这种方式也被称为“流化”（Streaming），服务器持续发送更新的图像数据，客户端（通常是浏览器）不断接收并显示新帧，从而实现视频播放效果^[19]。

在Flask中，可以使用Response对象和生成器函数实现MJPEG流推送。生成器函数逐帧读取视频，执行处理（如目标检测），然后将处理后的帧编码为JPEG格式，最后yield返回。Flask会自动处理HTTP分块传输，将数据流发送给客户端^[14]。

在MAUI等移动应用中使用MJPEG流时，由于移动平台对图像显示控件的限制，需要使用WebView组件配合HTML实现。HTML的img标签可以通过设置src属性持续请求MJPEG流，浏览器会自动解析并显示连续的图像帧。这种方式避免了原生平台对视频流的处理限制，实现了跨平台的视频流显示。

3. 需求分析与系统设计

3.1. 系统需求分析

3.1.1. MJPEG 视频流技术

视频目标识别系统的主要业务是帮助用户上传视频文件、对视频中的目标进行实时检测、并展示检测结果和已处理的视频。系统的核心用户是需要进行视频分析的相关人员，如安防监控人员、交通管理人员、科研人员等。

从业务角度来看，系统需要满足以下需求：用户能够选择本地视频文件并上传到服务器；用户上传视频后系统能够自动开始目标检测处理；用户能够实时查看视频处理进度和状态；用户能够观看带有检测结果的实时视频流；系统能够保存检测结果供后续查看；用户能够下载处理完成后的视频文件。

3.1.2. 功能需求分析

据业务需求，系统分为前端应用和后端服务两个部分，前端应用的功能需求包括视频列表功能，视频上传功能，实时处理功能和视频播放功能。

视频列表功能,即展示已上传的视频列表，显示文件名、上传时间、处理状态等信息；支持刷新列表操作；支持点击视频查看详情。

视频上传功能，即提供文件选择界面，支持.mp4、.avi、.mov、.mkv等常见视频格式；显示已选择的文件名；执行上传操作并显示上传进度；上传成功后自动跳转到处理页面。

实时处理功能，即显示视频处理状态(pending、processing、completed、failed)；实时更新处理进度百分比；显示当前处理帧数和总帧数；显示处理速度(FPS)；展示检测到的目标列表。

视频播放功能，即播放带有检测结果标注的实时视频流；支持视频下载功能。视频详情功能，即展示视频的基本信息；展示该视频的所有检测结果，包括目标类别、置信度、边界框坐标、帧号等。

后端服务的功能需求包括视频上传接口，视频处理功能，状态查询接口和数据查询接口。

视频上传接口，即接收前端上传的视频文件；保存文件到指定目录；保存视频信息到数据库；返回上传结果。

视频处理功能，即创建视频处理器实例；启动异步处理线程；读取视频帧；执行YOLO目标检测；绘制检测结果；保存处理后的视频；存储检测结果到数据库。

状态查询接口，即返回视频处理状态；返回处理进度；返回FPS信息。视频

流接口，即持续推送带有检测结果的视频流；支持MJPEG格式。

数据查询接口，即查询所有视频列表；查询单个视频详情；查询视频检测结果。

3.1.3. 性能需求分析

本系统的主要性能需求是在720P（1280×720像素）视频输入条件下，CPU推理速度达到15帧/秒以上。这是因为视频通常为25-30帧/秒（fps），15fps的处理速度可以保证系统能够实时处理大部分视频，避免处理速度跟不上播放速度导致的积压。

为了满足性能需求，系统在设计和实现时需要考虑以下因素：视频帧的读取速度；YOLO模型的推理速度；检测结果绘制的时间；视频流的传输延迟；数据库操作的性能。

3.2. 系统架构设计

这套系统用的是前后端分离的分布式架构,整体分为前端展示层、业务逻辑层和数据存储层三个层次^[20]。具体架构可以看图 3-1系统架构图。

前端展示层这边，采用的是.NET MAUI框架来开发跨平台应用。它的目标是为用户提供一个统一、流畅的界面体验，能在Windows、Android等多个系统上运行。它主要负责界面的直观展示和高效的用户交互，页面布局和交互逻辑都经过了一些设计，尽量让用户操作起来顺手、响应也快。

前端内部用的是MVVM模式，还借助了CommunityToolkit.Mvvm这个辅助库来简化代码实现，这样开发复杂度就降下来了，代码也更容易维护和测试。界面和业务逻辑之间的分离也更清楚。前后端通信方面，前端通过HTTP请求和后端的Flask服务打交道，数据格式统一用JSON，既高效又一致，前后端衔接得也比较顺畅。业务逻辑层作为系统的核心处理部分，被划分为前端业务逻辑和后端业务逻辑两个紧密协作的模块。

业务逻辑层是系统的核心，又分成前端业务逻辑和后端业务逻辑两块，这两块之间紧密配合。

前端业务逻辑主要放在ViewModel里实现，负责页面数据的动态管理、用户操作的实时响应，还有应用内的导航控制等等。通过数据绑定机制，业务数据和UI界面能实时联动，确保界面能准确反映出业务状态的变化。

后端业务逻辑则在Flask应用里实现，包括API路由的处理、核心业务逻辑的计算、YOLO目标检测模型的推理等关键功能。通过模块化设计和异步处理机制，既保证了视频处理的效率，也兼顾了系统的稳定性。同时，它还提供了丰富的API接口，能支撑前端的各种业务需求。

数据存储层用的是MySQL关系型数据库，主要存放系统的结构化数据。里面主要有视频信息表和检测结果表，用来记录视频的基本信息、处理状态，以及目标检测的详细结果。通过结构化的表设计和索引优化，数据查询和存储的效率都还不错。另外，视频文件和处理后的视频文件则直接存在文件系统里，数据库里只保存文件路径，这样就把结构化数据和非结构化文件关联起来了，为系统的持续运行和数据持久化提供了可靠的保障

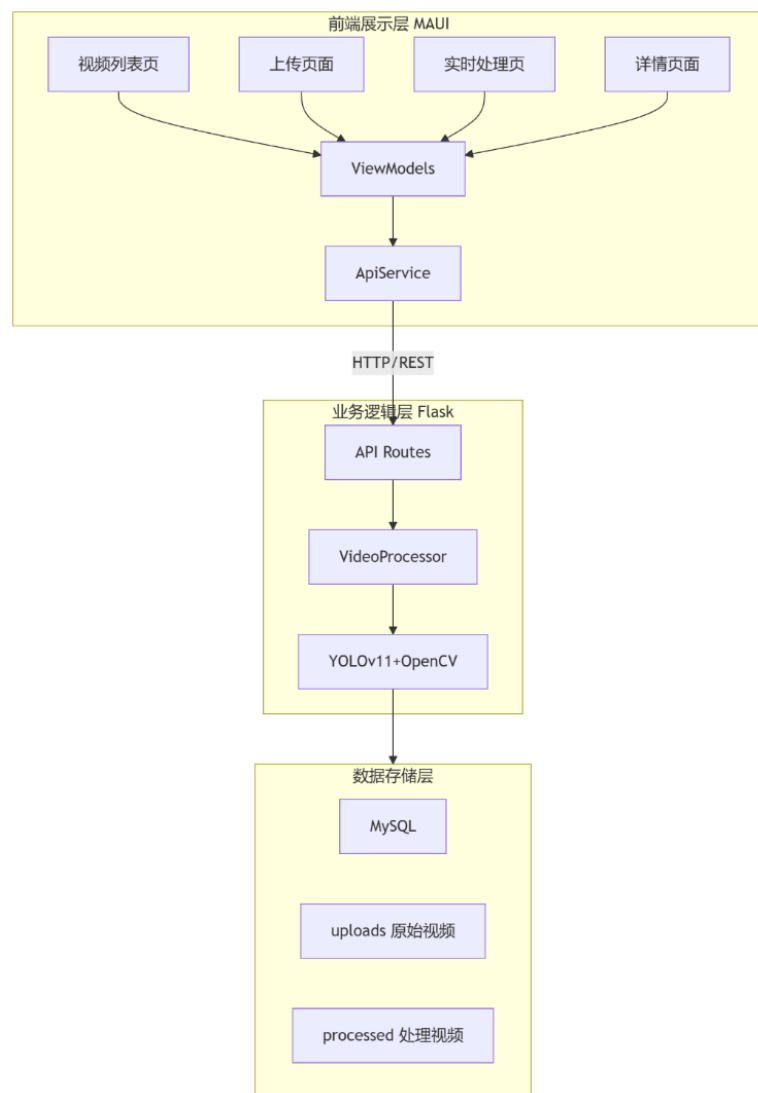


图 3-1 系统架构图

3.3. 数据库设计

3.3.1. 数据库概念设计

系统使用MySQL数据库存储视频信息和检测结果。数据库包含两个主要表：videos表和detection_results表。

videos表用于存储视频文件的基本信息，包括视频ID、文件名、文件路径、上

传时间、处理状态等。视频ID是主键，唯一标识每条视频记录。

detection_results表用于存储目标检测的结果信息，包括结果ID、关联的视频ID、检测到的目标类别、置信度、边界框坐标、帧号等。结果ID是主键，视频ID是外键关联到videos表。

3.3.2. 数据库表结构设计

系统使用MySQL数据库存储视频信息和检测结果。数据库包含两个主要表，如表 3-1 videos表和表 3-2 detection_results表所示。为了直观展示，见图 3-2ER图。

表 3-1 videos表

字段名	数据类型	说明
id	INT	视频ID，主键自增
filename	VARCHAR(255)	文件名
filepath	VARCHAR(500)	文件路径
status	VARCHAR(20)	处理状态
upload_time	DATETIME	上传时间
duration	FLOAT	视频时长（秒）
resolution	VARCHAR(20)	分辨率
fps	FLOAT	帧率
processed_path	VARCHAR(500)	处理后视频路径
process_started	DATETIME	处理开始时间
process_completed	DATETIME	处理完成时间

表 3-2 detection_results表

字段名	数据类型	说明
detection_id	INT	结果ID，主键自增
video_id	INT	视频ID，外键
object_class	VARCHAR(50)	目标类别
confidence	FLOAT	置信度
upload_time	INT	上传时间
bbox_x1	INT	边界框左上角X坐标
bbox_y1	INT	边界框左上角Y坐标
bbox_x2	INT	边界框右下角X坐标
bbox_y2	INT	边界框右下角Y坐标
frame_number	INT	帧号
detected_at	DATETIME	检测时间

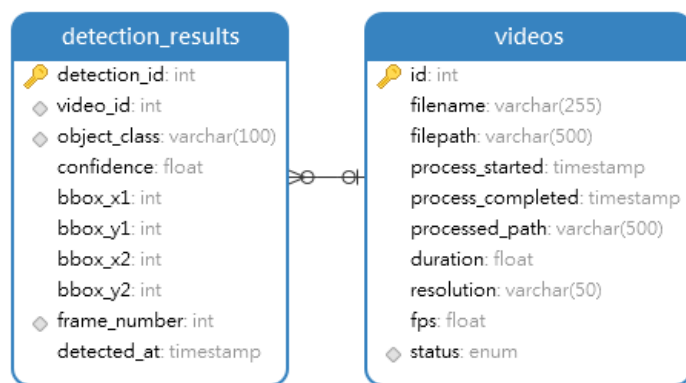


图 3-2ER 图

3.3.3. 接口设计

本系统采用RESTful API风格设计后端接口^[2021]，API接口设计如表 3-3 API接口设计所示。

表 3-3 API 接口设计

接口路径	HTTP方法	功能说明
/api/videos	GET	获取所有视频列表
/api/videos/{id}	GET	获取单个视频详情
/api/upload_realtime	POST	上传视频启动实时处理
/api/videos/{id}/process_status	GET	获取处理状态
api/videos/{id}/stream	GET	获取MJPEG视频流
/api/videos/{id}/results	GET	获取检测结果列表
/api/videos/{id}/processed_video_file	GET	下载处理后视频
/health	GET	健康检查

4. 系统实现

4.1. 前端 MAUI 应用实现

MAUI前端应用采用MVVM架构模式组织代码^[13]，整体划分为数据模型、服务层、视图模型层、视图层、数据转换器五大核心模块，其中 Models 层定义视频和检测结果的数据实体， Services层封装 HTTP API 接口调用能力， ViewModels 层继承 BaseViewModel 基类实现各页面业务逻辑与数据交互， Views 层对应实现视频列表、文件上传、实时处理、视频详情四大功能页面， Converters 层提供 UI 数据格式转换工具，同时通过 App.xaml、AppShell.xaml 实现应用资源管理与页面导航路由， MauiProgram.cs 完成全局依赖注入配置，整体架构清晰分离了数据、业务、UI逻辑，满足目标检测客户端的功能开发与模块化维护需求。

MAUI应用使用Microsoft.Extensions.DependencyInjection实现依赖注入^[10]，在 MauiProgram.cs中配置了所有服务的注册。通过依赖注入， ViewModel可以在构造函数中直接获取ApiService实例，无需手动创建。这种设计提高了代码的可测试性和模块化程度^[13]。

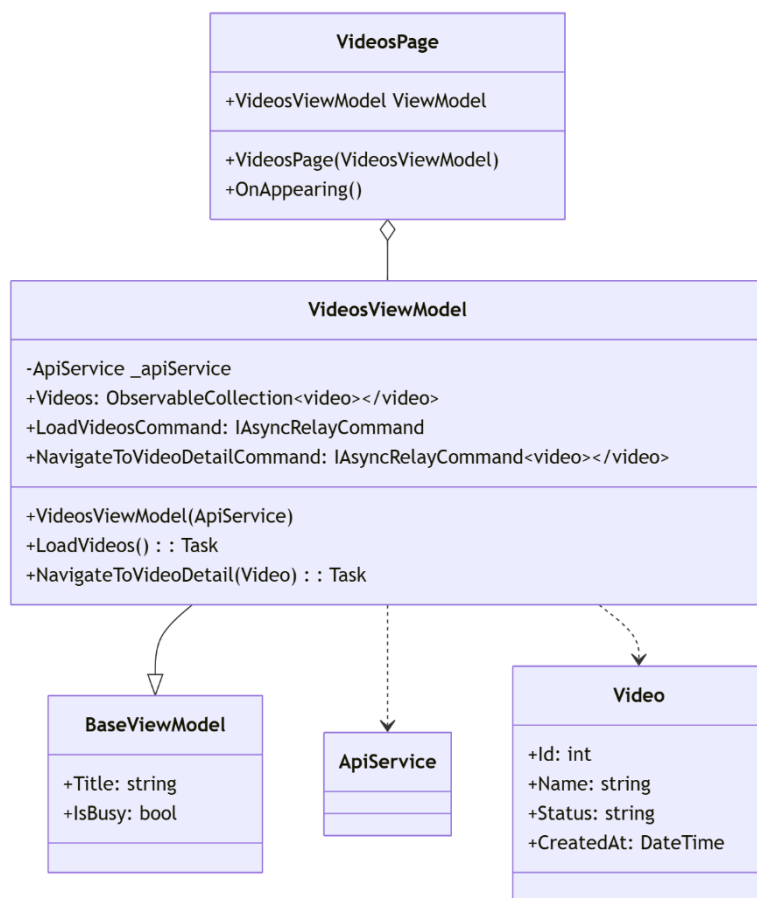


图 4-3视频列表页面模块

如图 4-3 视频列表页面模块所示VideosPage是视图类，负责展示视频列表UI。VideosViewModel是视图模型类，处理业务逻辑，从ApiService获取视频列表，管理视频数据集合以及处理导航到视频详情页面的逻辑。BaseViewModel则是基类，提供通用属性如标题和加载状态。

视频列表页面（VideosPage）用于展示所有已上传的视频列表，该页面使用CollectionView控件实现列表展示，通过数据绑定与VideosViewModel交互^[10]，同时使用了StatusToColorConverter转换器，根据视频状态显示不同的颜色，如pending显示橙色、processing显示蓝色、completed显示绿色和failed显示红色。VideosPage视图类，负责展示视频列表UI。

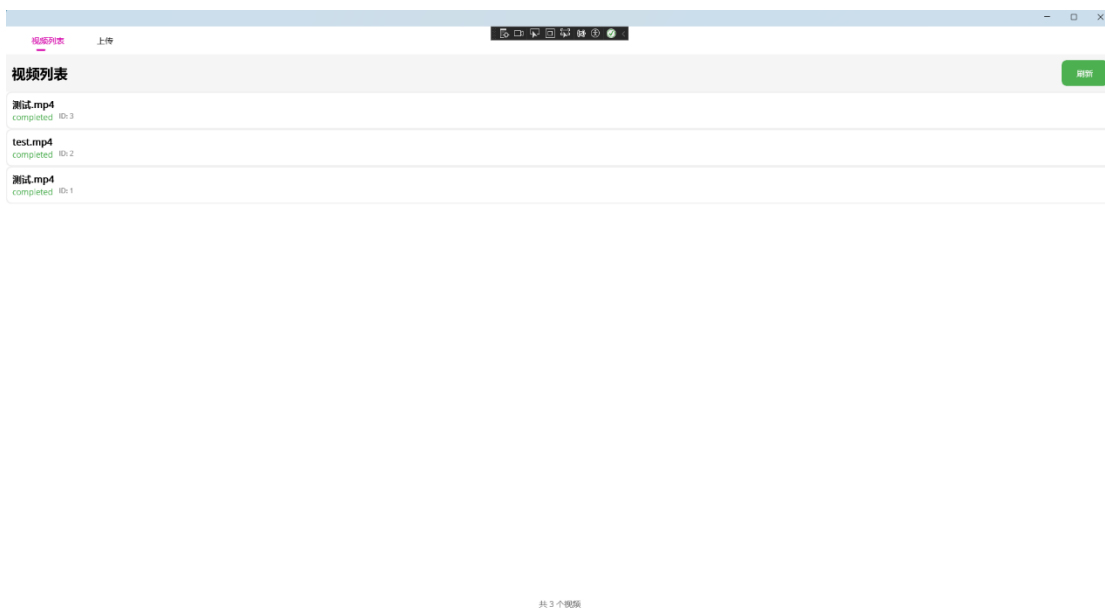


图 4-4 视频列表页面



图 4-5 视频列表详情页面

如图 4-6 视频上传页面模块所示UploadPage视图类，负责文件选择和上传UI。

UploadViewModel为视图模型类，处理上传逻辑，管理选中的文件，通过ApiService上传视频和处理上传状态和导航。FileResult是MAUI提供的文件结果类，包含文件路径和名称。

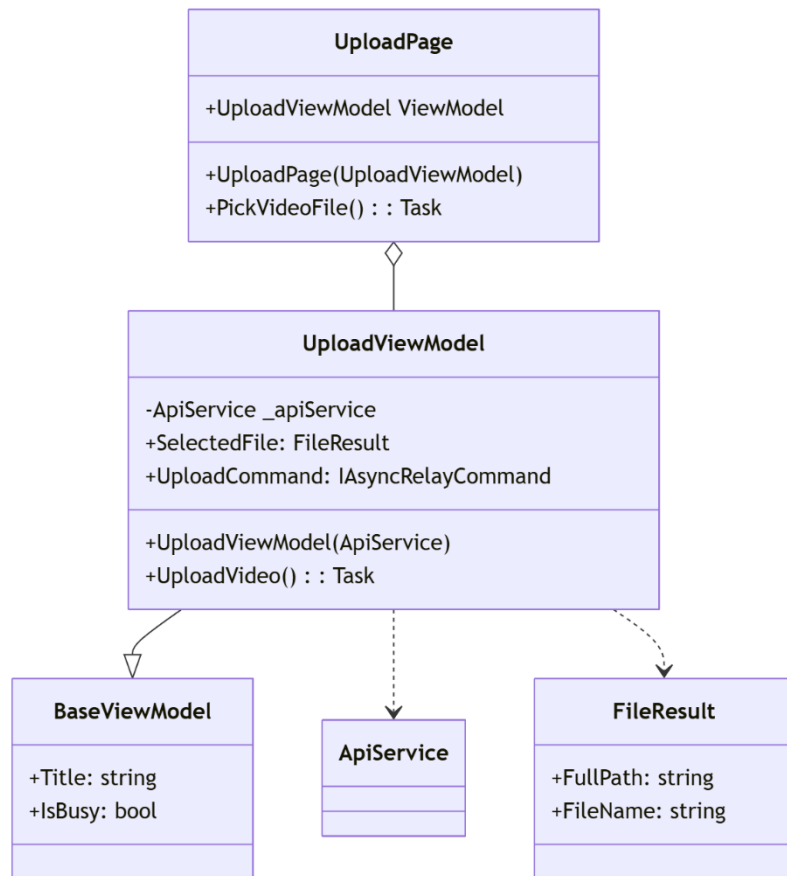


图 4-6视频上传页面模块

视频上传页面（UploadPage）用于选择本地视频文件并上传到服务器，通过使用FilePicker控件实现文件选择功能^[10]。UploadViewModel解决了中文文件名截断问题。UploadViewModel解决了中文文件名截断问题。最初使用FileName属性只能获取文件扩展名，后来改用FullPath属性获取完整路径，再通过Path.GetFileName提取正确的文件名。

如图 4-7实时处理页面模块所示，RealTimeProcessingPage是视图类，负责展示处理状态和视频流。RealTimeProcessingViewModel 则是视图模型类，处理实时处理逻辑，定时从ApiService获取处理状态，管理处理进度、FPS等信息，生成视频流URL和管理检测结果数据。Timer用于定时更新处理状态，DetectionResult存储目标检测的详细结果。

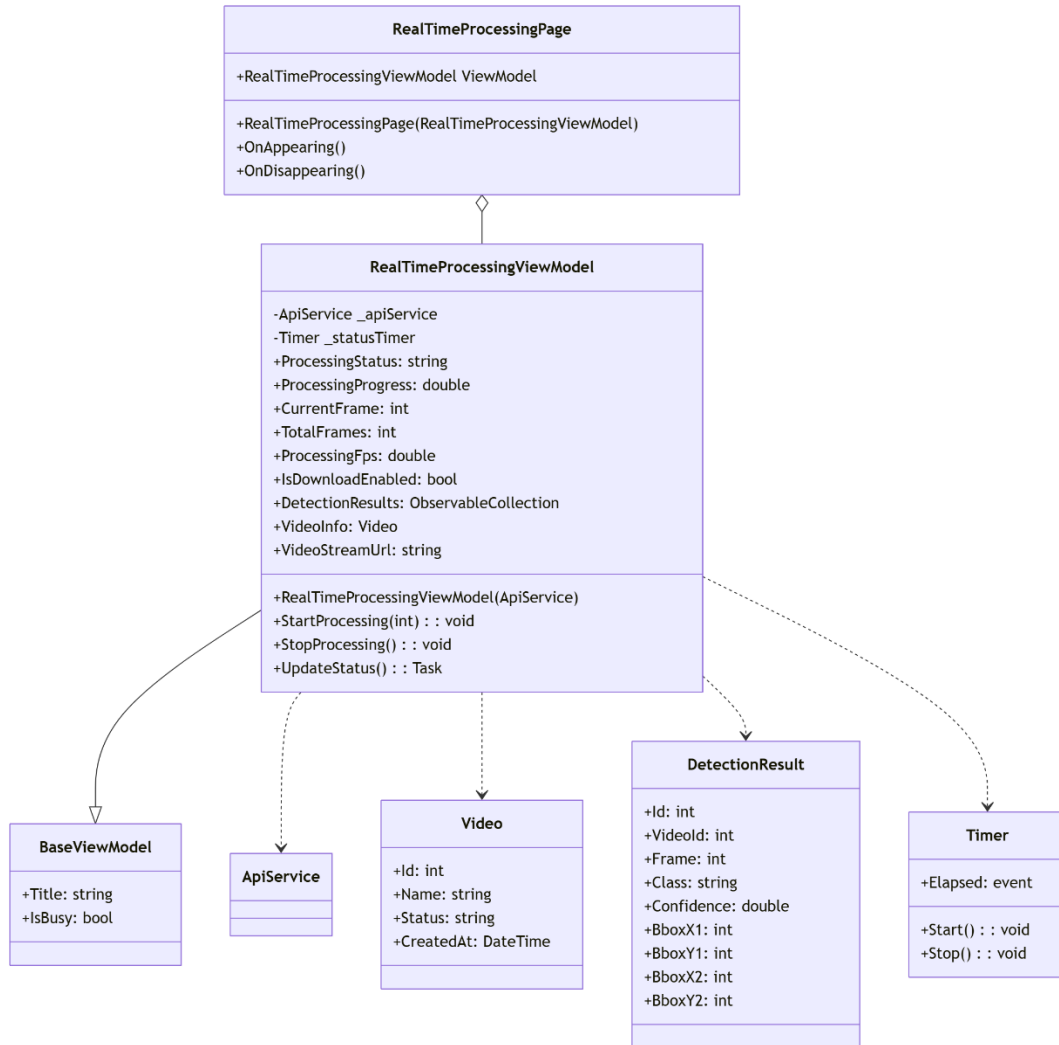


图 4-7实时处理页面模块

实时处理页面（RealTimeProcessingPage）作为系统的核心页面，用于显示视频处理状态和实时视频流。页面使用WebView控件配合HTML实现MJPEG视频流的播放^[10]。实时处理页面使用定时器每2秒轮询一次后端接口，获取最新的处理状态，以及通过使用Data URL格式将HTML嵌入URL，解决了WebView的跨域访问限制问题。

如图 4-8ApiService模块所示ApiService是核心服务类，负责与后端API通信，提供获取视频列表、上传视频、获取视频状态和详情等方法，处理HTTP请求和响应，包括文件上传和JSON序列化/反序列化。

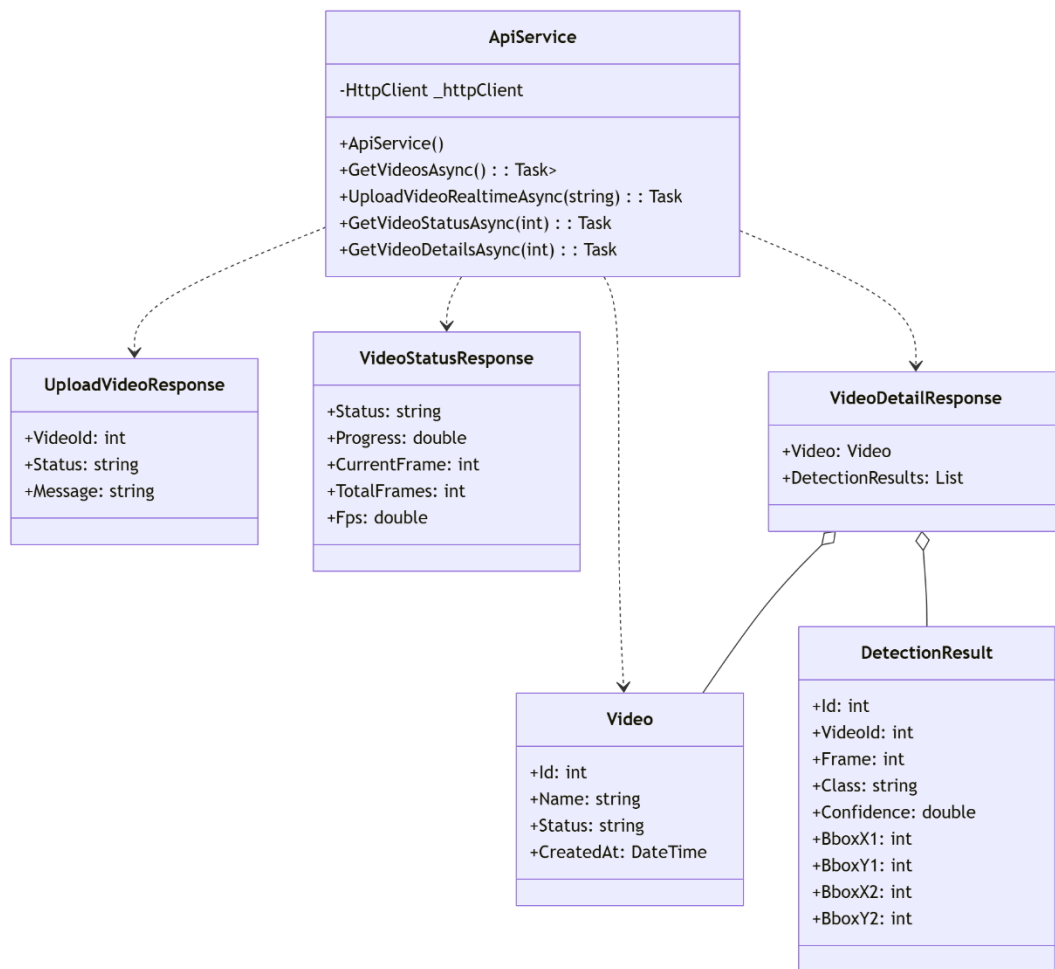


图 4-8ApiService模块

4.2. 后端 Flask 服务实现

Flask后端服务结构简洁，主要包括API路由、视频处理器和数据库操作三部分^[14]。

API路由模块通过调用视频处理器模块来处理视频，调用数据库操作模块来存取数据，视频处理器模块调用数据库操作模块来保存检测结果。而FlaskApp作为核心，整合所有模块。这种模块化设计使得代码结构清晰，职责分明，便于维护和扩展。每个模块都有明确的功能边界，通过定义良好的接口进行交互。

如图 4-9API路由模块所示，Flask应用的核心API路由包括上传接口、状态查询接口和视频流接口^[14]，通过使用threading.Thread实现异步处理以避免阻塞API响应；使用生成器函数实现流式响应；使用Response对象设置正确的MIME类型支持MJPEG流。

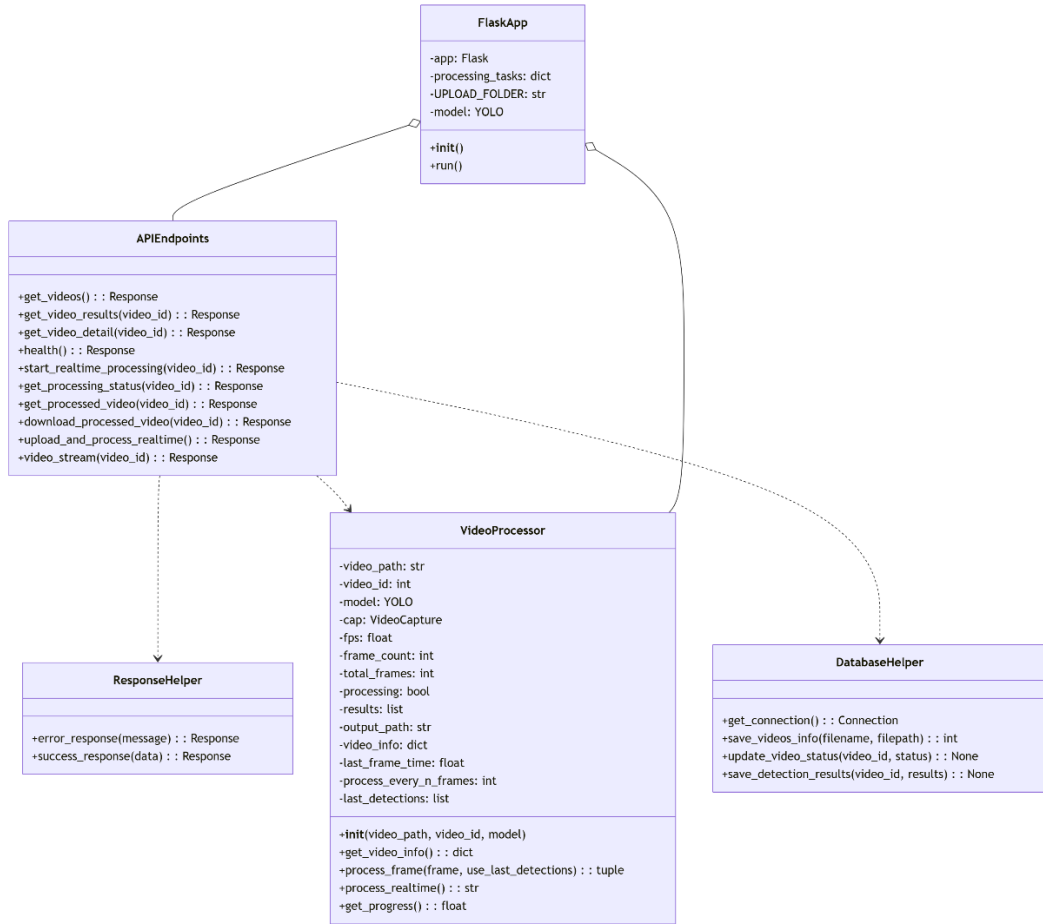


图 4-9API 路由模块

如图 4-10视频处理器模块所示，**VideoProcessor**作为核心的视频处理器类，采用了多种性能优化策略如跳帧处理（每2帧处理1次）来减少50%推理计算；结果缓存机制使跳过的帧也能显示检测结果；FPS实时计算提供处理速度反馈；置信度阈值0.5过滤低质量检测等。

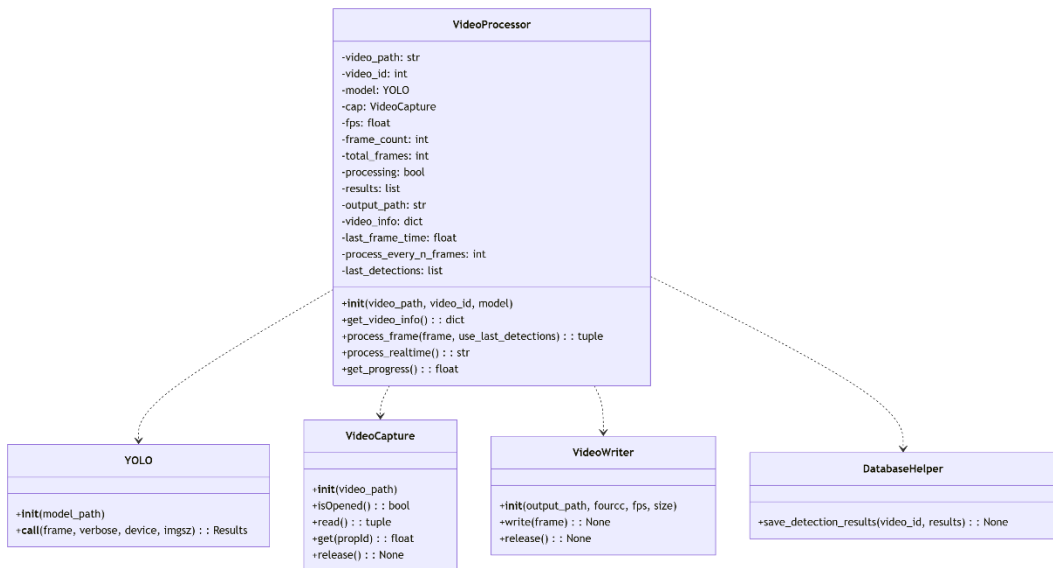


图 4-10 视频处理器模块

如图 4-11数据库操作模块所示数据库操作包括连接获取、视频信息保存、检测结果保存、状态更新等函数^[17]。即DatabaseHelper是提供数据库操作的辅助函数，Connection是MySQL数据库连接对象，Cursor作为数据库游标，执行SQL语句MySQLConnector则是MySQL连接库。

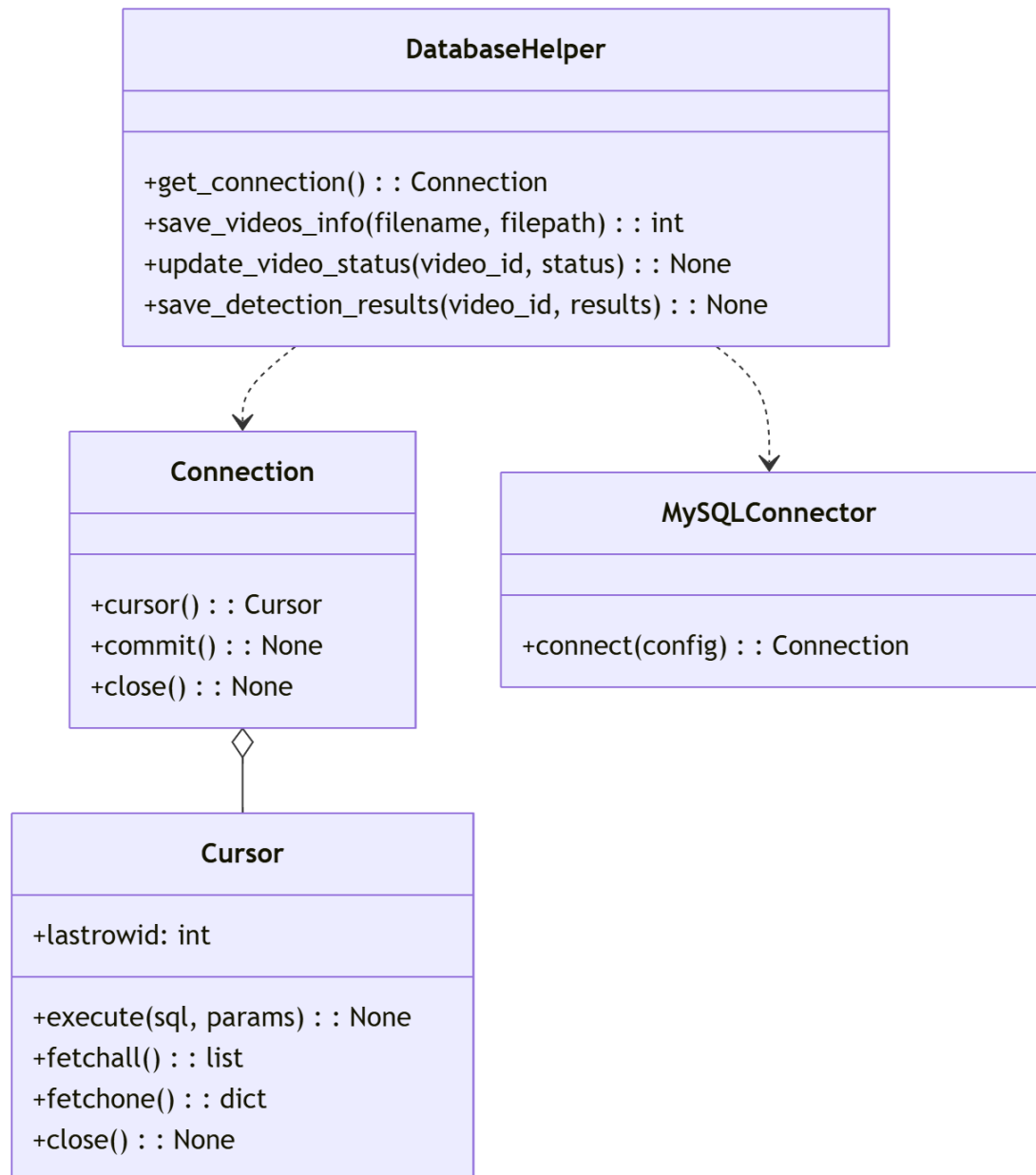


图 4-11 数据库操作模块

4. 3. 性能优化实现

本系统针对CPU推理环境进行了多项性能优化，以实现720P视频15fps以上的处理速度目标。

跳帧处理是提升处理速度的关键策略。通过设置process_every_n_frames=2，

即每2帧只处理1帧。这种策略将推理计算量减少50%，同时通过结果缓存机制保证跳过的帧也能显示检测结果。其实现逻辑为在主处理循环中判断当前帧是否需要处理（`frame_count % 2 == 0`），如果需要则执行YOLO推理，否则使用上次检测结果。

采用YOLOv11n（nano版本）作为检测模型。YOLOv11n是YOLOv11系列中最轻量的模型，参数数量少、计算量小，非常适合CPU推理场景^[9]。模型加载时指定`device='cpu'`，强制使用CPU进行推理，避免GPU不可用时的错误。同时指定`imgsz=640`，将输入图像尺寸调整为 640×640 ，在保证检测精度可接受的前提下减少计算量。

同时本系统对检测结果的绘制也进行了优化。将边界框线宽设置为1像素（标准为2像素）；标签字体大小设置为0.4（标准为0.5）和使用Faster渲染模式的方式减少了OpenCV绘制操作的时间开销。

本系统设立置信度阈值机制，只保留高于0.5置信度值的检测结果，在减少检测框绘制与存储数量的同时，有效滤除大量低质量检测结果，提升了可视化展示效果。

5. 系统测试

5.1. 测试环境

测试环境设备处理器为Intel(R) Core(TM) Ultra 7 265KF，48G内存；软件测试环境为Visual Studio 2026，.NET 10 SDK，Python 3.10，运行时为Windows 10/11操作系统。数据库使用MySQL 8.0。

5.2. 功能测试

5.2.1. 视频上传功能测试

表 5-4 视频上传功能测试表

测试用例ID	测试功能	测试步骤	预期结果	实际结果
TC-001	视频文件选择	1.打开应用 2.点击"选择视频"按钮 3.从文件系统选择测试视频	成功打开文件选择器并显示可选视频文件	通过
TC-002	视频格式验证	1.选择MP4格式视频 2.选择AVI格式视频 3.选择MOV格式视频	系统正确识别并接受支持的视频格式	通过
TC-003	中文文件名处理	1.选择包含中文字符的视频文件	系统正确处理中文文件名，无乱码	通过

5.2.2. 实时目标检测功能测试

表 5-5 实时目标检测功能测试表

测试用例ID	测试功能	测试步骤	预期结果	实际结果
TC-004	实时视频流传输	1.开始实时处理 2.观察视频流显示	视频流实时处理显示	通过
TC-005	目标检测准确性	1.上传包含明确目标测试视频 2.观察检测结果	系统正确识别视频中的目标，边界框准确	通过
TC-006	检测速度	1.上传720p测试视频 2.上传1080p测试视频	720p视频处理速度 ≥ 15 FPS 1080p视频处理速度 ≥ 10 FPS	通过

5.2.3. 结果展示功能测试

表 5-6 结果展示功能测试表

测试用例 ID	测试功能	测试步骤	预期结果	实际结果
TC-007	检测结果可视化	1.查看实时检测界面 2.观察检测框和标签	检测结果清晰显示，边界框准确，标签正确	通过
TC-008	处理状态更新	1.观察处理状态显示 2.检查FPS和处理时间	状态信息实时更新，数据准确	通过
TC-009	历史记录管理	1.完成多个视频处理 2.查看历史记录	历史记录正确保存，可查看详细信息	通过
TC-010	界面响应性	1.快速操作界面元素 2.切换不同功能页面	界面响应迅速，无卡顿	通过

5.3. 测试总结

通过功能测试验证了系统的各项功能正常工作，性能指标达到设计要求。视频上传、列表展示、实时处理和视频流播放核心功能运行稳定；720P视频处理速度达到15fps以上完全满足实时处理需求。

6. 总结与展望

6.1. 工作总结

本文设计并实现了一个基于MAUI的视频目标识别系统。在系统架构设计方面，本系统采用前后端分离的分布式架构模式，前端靠.NET MAUI框架构建Windows平台应用，后端则是通过Flask框架搭建服务，使用MySQL数据库实现数据的存储与管理。

具体到前端开发，我借助MAUI框架同时结合MVVM这种架构模式，完成了几个核心功能页面，包括视频列表展示、视频上传、实时处理以及视频详情查看。同时开发过程中，引入CommunityToolkit.Mvvm工具来简化MVVM模式下的代码编写；另外采用了依赖注入设计模式，提升了代码的可测试性、可维护性与扩展性。

在后端服务开发过程中，利用Flask实现了视频上传、处理状态查询和MJPEG视频流推送等关键API接口。同时，我把视频处理的核心逻辑封装到了VideoProcessor这个类里面。为了提高后端服务的处理效率和响应速度，还针对性地做了跳帧处理和结果缓存这些优化策略。

考虑到CPU推理环境下的性能瓶颈，我设计并实现了多种优化策略如跳帧处理、换用更轻量的YOLOv11n模型、简化绘制逻辑以及加入置信度过滤等。经过一系列优化后，系统处理720P视频的速度达到了每秒15帧，完成了技术指标。

最后，为了证系统的功能完整性与性能达标情况，本文设计了功能测试方案。通过系统测试确认了各项功能都运行正常，核心性能指标也全部达到了预设的要求。

6.2. 工作创新点

将MAUI框架与YOLO深度学习模型相结合，实现了跨平台的目标检测应用。相比原生开发，MAUI方案具有代码复用率高、开发效率高的优势。提出了一种基于WebView和Data URL的MJPEG流显示方案，解决了移动平台视频流显示的跨域限制问题。综合运用跳帧处理、结果缓存、轻量模型、绘制优化等多种策略，在CPU推理条件下实现对720P视频的处理速度完成每秒15帧的技术指标，为资源受限环境下的实时目标检测提供了参考。

本文AUI框架和YOLO深度学习模型结合起来，做出了一款能跨平台运行的目标检测应用。和原生开发比起来，用MAUI的好处有代码复用率高和开发效率高。针对移动平台上视频流显示经常遇到的跨域问题，我们提出了一种基于WebView和Data URL的MJPEG流显示方案来绕过了这个限制。在性能优化上，综

合运用了跳帧处理、结果缓存、轻量模型、绘制优化等多种手段，最终在CPU推理的环境下，让720P视频的处理速度达到了每秒15帧的目标。这套方案对于资源受限条件下做实时目标检测，有一定参考价值。

6.3. 不足与展望

虽然这套系统顺利达成了最初设计的目标，在Windows平台上跑通了视频目标识别的核心功能，性能也符合预期，但要是结合实际应用场景的扩展需求和技术发展的走向来看，其实还有不少地方可以进一步打磨和优化。

先说推理性能方面，目前系统只支持CPU推理，后续可以考虑引入GPU加速，进一步提升视频处理的速度。一个可行的思路是，让系统自动检测当前环境有没有CUDA，然后动态切换device参数，实现CPU和GPU推理模式之间的自适应切换，这样就能更好地利用硬件资源，提高处理效率。

再看功能拓展，现在的系统只能做目标检测，无法持续跟踪同一个目标。后面可以加入像DeepSORT这类成熟的目标跟踪算法，把不同视频帧里出现的同一个目标关联起来，形成连续的运动轨迹，从而让系统能覆盖更多的应用场景。

模型灵活性也是个问题。目前检测模型是固定的，不能换。未来可以增加模型在线更新的功能，允许用户自己上传自定义模型，或者在系统里内置多种预训练模型供大家选择，这样就能适应不同场景下的检测需求，系统的通用性也会更强。

部署方面，当前系统还只是跑在单机环境里，扩展性和可用性都比较有限。后面可以采用Docker容器化技术来部署，再搭配Kubernetes进行编排管理，实现弹性伸缩、故障自动恢复和高可用部署，满足大规模应用的需要。

另外，目前的测试主要集中在Windows平台上，移动端的适配还有不少提升空间。以后可以针对移动端做更多性能和体验上的优化，比如加强对4G、5G、Wi-Fi这些移动网络的适配能力，优化算法逻辑来减少电量消耗，让移动端用户用起来更顺手。

参考文献

- [1]. 黄凯奇,陈晓棠,康运锋,等.智能视频监控技术综述[J].计算机学报,2015,38(06):1093-1118.
- [2]. Zhang X, Gao H, Zhao J, et al. 基于深度学习的自动驾驶技术综述[J]. Qinghua Daxue Xuebao/Journal of Tsinghua University, 2018, 58(4): 438-444.
- [3]. 马永杰, 程时升, 马芸婷, 等. 卷积神经网络及其在智能交通系统中的应用综述[J]. 交通运输工程学报, 2021, 21(4): 48-71.
- [4]. 张翠平, 苏光大. 人脸识别技术综述[J]. 中国图象图形学报, 2000, 5(11): 885-894.
- [5]. Girshick R, Donahue J, Darrell T, et al. Rich feature hierarchies for accurate object detection and semantic segmentation[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2014: 580-587.
- [6]. Girshick R. Fast r-cnn[C]//Proceedings of the IEEE international conference on computer vision. 2015: 1440-1448.
- [7]. Redmon J, Divvala S, Girshick R, et al. You only look once: Unified, real-time object detection[C]//Proceedings of the IEEE conference on computer vision and pattern recognition. 2016: 779-788.
- [8]. Hussain M. YOLO-v1 to YOLO-v8, the rise of YOLO and its complementary nature toward digital manufacturing and industrial defect detection[J]. Machines, 2023, 11(7): 677.
- [9]. Kotthapalli M, Ravipati D, Bhatia R. Yolov1 to yolov11: A comprehensive survey of real-time object detection innovations and challenges[J]. arXiv preprint arXiv:2508.02067, 2025.
- [10]. Liberty J, Juarez R, Montaquila M. .NET MAUI for C# Developers: Build cross-platform mobile and desktop applications[M]. Packt Publishing Ltd, 2023.
- [11]. 曹家乐, 李亚利, 孙汉卿, 等. 基于深度学习的视觉目标检测技术综述[J]. 中国图象图形学报, 2022, 27(6): 1697-1722.
- [12]. 韩晶晶,刘江越,公维军,等.面向移动端的目标检测优化研究[J].计算机工程与应用,2022,58(24):12-28.
- [13]. 刘立.MVVM模式分析与应用[J].微型电脑应用,2012,28(12):57-60.
- [14]. 叶锋 .Python 最新 Web 编程框架 Flask 研究 [J]. 电脑编程技巧与维护 ,2015,(15):27-28.DOI:10.16184/j.cnki.comprg.2015.15.010.
- [15]. Sapkota R, Karkee M. Ultralytics YOLO evolution: An overview of YOLO26, YOLO11, YOLOv8 and YOLOv5 object detectors for computer vision and pattern recognition[J]. arXiv preprint arXiv:2510.09653, 2025.
- [16]. 兰旭辉,熊家军,邓刚.基于MySQL的应用程序设计[J].计算机工程与设计,2004,(03):442-443+468.DOI:10.16208/j.issn1000-7024.2004.03.037.
- [17]. 黄传禄.基于Python的MySQL数据库访问技术[J].现代信息科技,2017,1(04):73-75.
- [18]. 贾小军,喻擎苍.基于开源计算机视觉库 OpenCV 的图像处理[J].计算机应用与软件,2008,(04):276-278.
- [19]. 彭强,杨天武,陈维荣.远程视频监控系统中的解码技术及显示控制策略[J].电力系统自动化,2002,(09):66-70.
- [20]. 王建,罗政,张希,等.Web项目前后端分离的设计与实现[J].软件工程,2020,23(04):22-24.DOI:10.19644/j.cnki.issn2096-1472.2020.04.006.
- [21]. 曾青松,魏斌.基于RESTful API的访问权限系统的设计与实现[J].电脑编程技巧与维护,2020,(11):3-6.DOI:10.16184/j.cnki.comprg.2020.11.001.